

AWS Hands-on Lab

Give an EC2 Instance Access to S3 Using an IAM Role (No Access Keys)

1. The Problem

A program running on an EC2 instance often needs other AWS services, for example to save files in S3. The wrong way is to create an IAM user, generate access keys, and paste them onto the server. Keys can leak (in code, logs, GitHub), never expire by themselves, and are hard to rotate.

The right way is an **IAM role attached to the instance**. AWS then hands the instance **temporary credentials** that rotate automatically. Nothing secret is stored on the server.

2. How It Works

- 1 You create a **role** whose **trust policy** says: the EC2 service may assume me.
- 2 You attach a **permissions policy** to the role (what it can do, e.g. read and write one bucket).
- 3 You attach the role to the instance through an **instance profile** (the console creates this for you).
- 4 Software on the instance (AWS CLI, boto3) automatically finds the credentials from the **instance metadata service** (169.254.169.254) and uses them. They are refreshed before they expire.

Part	Answers the question	In this lab
Trust policy	Who can assume the role?	ec2.amazonaws.com
Permissions policy	What can the role do?	List, read and write one bucket only
Instance profile	How does EC2 get the role?	Created automatically by the console

3. Lab Steps

Step A: Create a bucket

- 1 S3 > **Create bucket**. Name it something globally unique, e.g. yourname-ec2-role-lab-1234.
- 2 Keep **Block all public access** ON. Create the bucket and note its exact name and Region.

Step B: Create a least-privilege policy

IAM > Policies > Create policy > JSON tab. Replace BUCKET with your bucket name. Name it EC2-Lab-Bucket-Access.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListTheBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::BUCKET"
    },
    {
      "Sid": "ReadWriteObjects",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::BUCKET/*"
    }
  ]
}
```

Why two statements? Bucket-level actions (ListBucket) use the bucket ARN. Object-level actions (GetObject, PutObject) use the bucket ARN plus /*. Mixing them up is the most common S3 policy mistake. DeleteObject is deliberately not allowed.

Step C: Create the role

- 1 IAM > Roles > **Create role**. Trusted entity type: **AWS service**, use case: **EC2**.
- 2 Attach the EC2-Lab-Bucket-Access policy.
- 3 Name the role EC2-S3-Lab-Role and create it. Open it and look at the **Trust relationships** tab to see the ec2.amazonaws.com trust policy.

Step D: Attach the role to an instance

- 1 Use an existing instance or launch a new Free Tier eligible one (Amazon Linux 2023 includes the AWS CLI).
- 2 Existing instance: select it > **Actions** > **Security** > **Modify IAM role** > choose EC2-S3-Lab-Role > Update. New instance: Advanced details > IAM instance profile.
- 3 Connect with **EC2 Instance Connect** (select instance > Connect).

Step E: Test from inside the instance

```
aws sts get-caller-identity
# Arn shows: assumed-role/EC2-S3-Lab-Role/i-xxxxxxx (no IAM user, no keys)

echo "hello from ec2" > test.txt
aws s3 cp test.txt s3://BUCKET/          # allowed (PutObject)
aws s3 ls s3://BUCKET/                  # allowed (ListBucket)
aws s3 cp s3://BUCKET/test.txt back.txt # allowed (GetObject)

aws s3 rm s3://BUCKET/test.txt           # DENIED (no DeleteObject)
aws s3 ls                                # DENIED (no ListAllMyBuckets)
```

The two denied commands are the point of the lab: the role can do **exactly** what the policy says and nothing more.

Step F (optional): See the temporary credentials

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 300")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/iam/security-credentials/
# prints the role name. Add /EC2-S3-Lab-Role to the URL to see
# AccessKeyId, SecretAccessKey, Token and an Expiration time.
```

This uses **IMDSv2** (token first, then request), the more secure version. Never paste the credential output anywhere or commit it to GitHub. It expires within hours, but treat it as secret.

4. Reading Errors

Symptom	Likely cause
Unable to locate credentials	No role attached to the instance (check Actions > Security > Modify IAM role).
AccessDenied on PutObject	Policy resource is missing /* or the bucket name is misspelled.
AccessDenied on ls (no bucket name)	Role lacks s3:ListAllMyBuckets. Expected in this lab.
Works in console but not CLI	The console uses your IAM user; the CLI on the instance uses the role. They have different permissions.
Wrong Region errors	Add --region to the command or set the default Region.

5. Key Takeaways

- Use **roles, not access keys**, for AWS services that need AWS access.
- A role needs a **trust policy** (who) and a **permissions policy** (what).
- Credentials from roles are **temporary and rotate automatically**.
- Bucket actions use the bucket ARN; object actions use bucket/*.
- Least privilege: allow only the actions and resources needed. Anything not allowed is implicitly denied.

6. Check Yourself

Question	Answer
Why is a role safer than access keys on a server?	Credentials are temporary, rotate automatically and are never stored on disk or in code.
What does the trust policy of an EC2 role name as Principal?	The service ec2.amazonaws.com.
Which command shows which identity the instance is using?	aws sts get-caller-identity
Why did aws s3 rm fail?	The policy does not allow s3:DeleteObject, so it is implicitly denied.
Does attaching a role require stopping the instance?	No. It can be attached or changed while the instance is running.

7. Cleanup

- Empty and delete the S3 bucket.
- Detach the role from the instance (Actions > Security > Modify IAM role > No IAM role).
- Delete the role EC2-S3-Lab-Role and the policy EC2-Lab-Bucket-Access.
- Stop or terminate the instance if you launched it only for this lab.

Next lab ideas: EBS snapshots and creating your own AMI; Application Load Balancer with two instances; the same S3 upload from Python using boto3 with the instance role.